

```
[28]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
import math
import pandas
x=[]
myTable=[]
b=[]
for i in range(0,7):
    x.append(-5.0+((10.0/6.0)*i))
def f(x):
    return 1/(1+math.exp(-x))
for i in range(0,7):
    b.append(f(x[i]))
for i in range(0,7):
    myTable.append([x[i],b[i]])
head=["x","f(x)"]
print(pandas.DataFrame(myTable, columns=head))
a=np.vander(x)
solution=np.linalg.solve(a,b)
def vand(coefficients,x):
    solution=0
    for i in range(0,len(coefficients)):
        solution+=coefficients[i]*(x**(len(coefficients)-1-i))
    return solution
x1=[]
for i in range(0,101):
    x1.append(-5.0+(0.1*i))
y=[]
for i in range(0,101):
    y.append(f(x1[i]))
print()
print("Mean: %f Standard Deviation: %f"%(np.mean(y),np.std(y)))
```

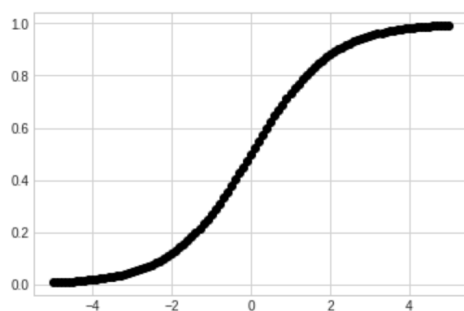
```
p6=[]
for i in range(0,101):
    p6.append(vand(solution,x1[i]))
errors=[]
for i in range(0,101):
    errors.append(abs(y[i]-p6[i])/abs(y[i]))
print()
print("Error: %f"%(max(errors)))
```

	x	f(x)
0	-5.000000	0.006693
1	-3.333333	0.034445
2	-1.666667	0.158869
3	0.000000	0.500000
4	1.666667	0.841131
5	3.333333	0.965555
6	5.000000	0.993307

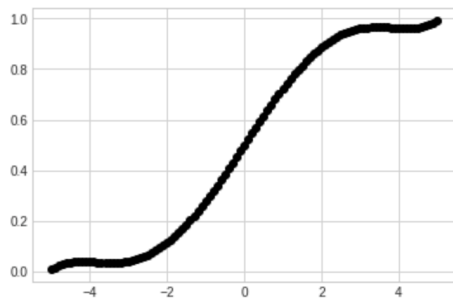
Mean: 0.500000 Standard Deviation: 0.390193

Error: 2.289838

```
[24]: y1=[]
for i in range(0,101):
    y1.append(f(x1[i]))
plt.plot(x1, y1, 'o', color='black');
```



```
[25]: y2=[]
      for i in range(0,101):
          y2.append(vand(solution,x1[i]))
      plt.plot(x1, y2, 'o', color='black');
```



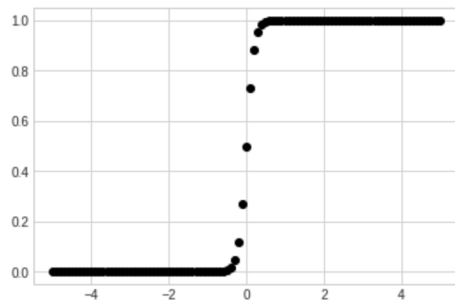
```
[29]: x=[]
      myTable=[]
      b=[]
      for i in range(0,7):
          x.append(-5.0+((10.0/6.0)*i))
      def f(x):
          return 1/(1+math.exp(-10*x))
      for i in range(0,7):
          b.append(f(x[i]))
      for i in range(0,7):
          myTable.append([x[i],b[i]])
      head=["x","f(x)"]
      print(pandas.DataFrame(myTable, columns=head))
      a=np.vander(x)
      solution=np.linalg.solve(a,b)
      def vand(coefficients,x):
          solution=0
          for i in range(0,len(coefficients)):
              solution+=coefficients[i]*(x**(len(coefficients)-1-i))
          return solution
      x1=[]
      for i in range(0,101):
          x1.append(-5.0+(0.1*i))
      y=[]
      for i in range(0,101):
          y.append(f(x1[i]))
      print()
      print("Mean: %f Standard Deviation: %f"%(np.mean(y),np.std(y)))
      p6=[]
      for i in range(0,101):
          p6.append(vand(solution,x1[i]))
      errors=[]
      for i in range(0,101):
          errors.append(abs(y[i]-p6[i])/abs(y[i]))
      print()
      print("Error: %f"%(max(errors)))
```

	x	f(x)
0	-5.000000	1.928750e-22
1	-3.333333	3.338238e-15
2	-1.666667	5.777748e-08
3	0.000000	5.000000e-01
4	1.666667	9.999999e-01
5	3.333333	1.000000e+00
6	5.000000	1.000000e+00

Mean: 0.500000 Standard Deviation: 0.489999

Error: 63101461467430133760.000000

```
[30]: y1=[]
      for i in range(0,101):
          y1.append(f(x1[i]))
      plt.plot(x1, y1, 'o', color='black');
```



```
[31]: y2=[]
      for i in range(0,101):
          y2.append(vand(solution,x1[i]))
      plt.plot(x1, y2, 'o', color='black');
```



For $\theta = 1$, the approximation is good other than a little bit of curvature towards the most negative and positive x values which is to be expected when using an interpolating polynomial. For $\theta = 10$, the approximation is much worse especially around $x=0$ because the true function moves almost asymptotically from $y=0$ to $y=1$ at $x=0$ while the interpolating polynomial moves very gradually from $y=0$ to $y=1$ around $x=0$. As we would expect based on this, the error is much, much larger for $\theta = 10$ than $\theta = 1$, showing that the interpolating polynomial is a better approximation for $\theta = 1$ than $\theta = 10$ in this specific case and a better approximation the smoother the curve is in general.